

MASARYKOVA UNIVERZITA
PŘÍRODOVĚDECKÁ FAKULTA
ÚSTAV MATEMATIKY A STATISTIKY

Bakalářská práce

BRNO 2026

FILIP FARKAŠOVSKÝ

Metody výběru prediktorů pro neuronové sítě

Bakalářská práce

Filip Farkašovský

Bibliografický záznam

Autor:	Filip Farkašovský Přírodovědecká fakulta, Masarykova univerzita Ústav matematiky a statistiky
Název práce:	Metody výběru prediktorů pro neuronové síte
Studijní program:	Matematika
Studijní obor:	Matematika (hlavní) – Ekonomie (vedlejší)
Vedoucí práce:	doc. Mgr. Jan Koláček, Ph.D.
Akademický rok:	2025/2026
Počet stran:	xii + 14
Klíčová slova:	Neurónové síte; Klíčové slovo; Klíčové slovo; Klíčové slovo; Klíčové slovo; Klíčové slovo; Klíčové slovo; Klíčové slovo

Bibliographic Entry

Author:	Filip Farkašovský Faculty of Science, Masaryk University Department of Mathematics and Statistics
Title of Thesis:	Feature selection methods for neural networks
Degree Programme:	Mathematics
Field of Study:	Mathematics (major) – Economics (minor)
Supervisor:	doc. Mgr. Jan Koláček, Ph.D.
Academic Year:	2025/2026
Number of Pages:	xii + 14
Keywords:	Neural networks; Keyword; Keyword; Keyword; Key- word; Keyword; Keyword; Keyword; Keyword

Abstrakt

V této bakalářské/diplomové/rigorózní práci se věnujeme ...

Abstract

In this thesis we study ...

ZADÁNÍ
BAKALÁŘSKÉ PRÁCE

Akademický rok: 2025/2026

Ústav:	Ústav matematiky a statistiky
Student:	Filip Farkašovský
Program:	Matematika
Specializace:	Ekonomie Matematika

Ředitel ústavu PřF MU Vám ve smyslu Studijního a zkušebního řádu MU určuje bakalářskou práci s názvem:

Název práce:	Metody výběru prediktorů pro neuronové sítě
Název práce anglicky:	Feature selection methods for neural networks
Jazyk práce:	angličtina

Oficiální zadání:

Cílem práce je přehledně představit metody výběru prediktorů se zaměřením na jejich využití v neuronových sítích. Součástí práce bude teoretický rozbor klasických i moderních přístupů, jejich vzájemné srovnání a praktická ukázka aplikace na vybraném datovém souboru.

Literatura:

GOODFELLOW, Ian; Yoshua BENGIO a Aaron COURVILLE. *Deep learning*. London, England: MIT Press, 2016, xxii, 775. ISBN 9780262035613.

LIU, Huan; Hiroshi MOTODA. *Computational Methods of Feature Selection*. Boca Raton, FL, USA: Chapman and Hall, 2007, 440 s. ISBN 9781584888789.

Vedoucí práce:	doc. Mgr. Jan Koláček, Ph.D.
Konzultant:	Mgr. Markéta Barač Makarová
Datum zadání práce:	3. 10. 2025
V Brně dne:	9. 11. 2025

Zadání bylo schváleno prostřednictvím IS MU.

Filip Farkašovský, 13. 10. 2025

doc. Mgr. Jan Koláček, Ph.D., 14. 10. 2025

RNDr. Jan Vondra, Ph.D., 14. 10. 2025

Poděkování

Na tomto místě bych chtěl(-a) poděkovat ...

Prohlášení

Prohlašuji, že jsem svoji bakalářskou/diplomovou práci vypracoval(-a) samostatně pod vedením vedoucího práce s využitím informačních zdrojů, které jsou v práci citovány.

Prohlašuji, že jsem svoji rigorózní práci vypracoval(-a) samostatně s využitím informačních zdrojů, které jsou v práci citovány.

Brno xx. měsíce 2026

.....
Filip Farkašovský

Contents

Notation	xv
Introduction	1
Kapitola 1. Theoretical Overview	3
1.1 Overview of Neural Networks	3
1.1.1 Brief History of Neural Networks	3
1.1.2 Network architectures	4
1.1.3 Notation	6
1.1.4 Activation functions in the Hidden Layer	6
1.1.5 Activation Functions in Output Layer	7
1.1.6 Loss Functions	8
1.1.7 Gradient Descent	8
1.2 Introduction to Feature Selection	10
1.3 Generalization in Deep Neural Networks	12
1.4 Evaluation of Model Performance	13
1.4.1 Criterias for classification tasks	13
1.4.2 Criteria for Regression Tasks	15
1.4.3 architerctures, crossvalidation, python pipeline, grid-search, hy- perparameters	16
Kapitola 2. Feature Selection Methods	17
Kapitola 3. Feature Selection Methods	19
3.1 Filter Methods	20
3.1.1 Subsection	21
3.2 Wrapper Methods	21
3.3 Embedded Methods	21
Kapitola 4. Practical application	23
4.1 Section	24
Conclusion	25
Appendix	27

Bibliography	29
Index	31

Notation

Pro snazší orientaci v textu zde čtenáři předkládáme přehled základního značení, které se v celé práci vyskytuje.

$\mathbb{C}$	množina všech komplexních čísel
$\mathbb{R}$	množina všech reálných čísel
$\mathbb{Z}$	množina všech celých čísel
$\mathbb{N}$	množina všech přirozených čísel
$\mathbb{C}$	množina všech komplexních čísel
$\mathbb{R}$	množina všech reálných čísel
$\mathbb{Z}$	množina všech celých čísel
$\mathbb{N}$	množina všech

Introduction

The neural network is a supervised machine learning model inspired by the biological neural networks of the human brain. Because of their ability to capture complex patterns, they are widely used in areas such as image recognition, language processing, and bioinformatics. However, their performance can decline when datasets contain irrelevant or redundant features. Large datasets with many features require longer training times and are harder to interpret, which is a problem often referred to as the *curse of dimensionality*. To cope with this problem, methods of feature selection are implemented.

Feature selection is the process of identifying the most relevant features in a dataset while removing those that do not contribute significantly to predictive performance. By reducing the number of input variables, we can improve model accuracy, speed up training, and make the results easier to interpret.

The main goal of this thesis is to explore different feature selection methods in the context of neural networks. The work will include a theoretical comparison of these approaches and a practical demonstration on a selected dataset, demonstrating how different methods influence model performance and interpretability.

The structure of the thesis is as follows: Chapter 1 discusses the importance of neural networks and feature selection. Chapter 2 introduces neural network fundamentals and basic concepts of feature selection. Chapter 3 reviews the main feature selection methods. Chapter 4 presents a practical application, including data pre-processing, model training, and evaluation.

This thesis combines theoretical review with practical experiments to demonstrate the importance of feature selection in neural networks and its impact on model performance.

The source code used in the practical part is included in the appendix and was implemented using Python libraries.

Chapter 1

Theoretical Overview

1.1 Overview of Neural Networks

Neural networks are one of the most popular algorithms of machine learning. Their were inspired by the structure and function of biological neurons, which are the fundamental units of nervous tissue. To distinguish them from their biological counterpart, sometimes the term *Artificial neural networks (ANNs)* is used.

Neural network can be viewed as a composition of affine transformations and non-linear activation function, which allows them to serve as universal approximators. This property is established in many theorems; several of which are mentioned by Goodfellow in [8]. Their ability to capture complex patterns makes them widely applicable in areas such as computer vision, natural language processing, and predictive modeling.

????? ?? ? including one relevant theorem for deep networks by Kidger and Lyons:

1.1.1 Brief History of Neural Networks

Origins: In 1949, Canadian psychologist Donald Hebb in [16] proposed that the connections (synapses) between neurons strengthen when they are activated together. This is often summarized by the phrase attributed to him: “neurons that fire together, wire together”. Hebbian theory provided a foundational idea, that networks could learn from experience.

Early models: A year later, the *Perceptron* created by Frank Rosenblatt in [17] became the first model capable of learning the weights that defined categories from examples. This model was a single-layer neural network, where each neuron multiplies inputs by adjustable weights. These weighted inputs were then summed up and passed through an activation function to produce a binary output. Though groundbreaking, it had many limitations due to its linearity. *Sidenote: Watching movies and documentaries is a valid research and you can cite them as well :) If you're lucky, some of these people even spoke about their research on TV, in public seminars, talks and such. I'd scout youtube for some which were uploaded publicly.*

Setback: The work of Minsky and Papert in [18] demonstrated that single-layer

linear networks could not solve problems such as the XOR function, which led to a temporary decline in research of neural networks [19]. [Reference to paragraph in this thesis discussing limitations of perceptron capturing logic of XOR function in this thesis \(maybe in similar way as it is showed by Goodfellow\)](#)

Revival: Neural networks regained attention with the development of multi-layer networks trained using backpropagation [20]. This allowed networks to learn more complex patterns and revived research in deep learning.

Specialised networks: In the 1990s researchers introduced specialized architectures for different types of data. The development of *convolutional neural networks*, as presented by LeCun, Bottou, Bengio, and Haffner in [21], for image processing, together with the use of *recurrent neural networks* for sequential data, enabled neural networks to tackle a wider variety of problems. This led to strong improvements in vision, speech, and language applications [22].

Developments up to 2026: Breakthroughs such as AlexNet [23] and the *transformer* architecture - built on the attention mechanism - [24] accelerated progress in image recognition and natural language processing. *The attention mechanism* allows a neural network to focus on the most relevant parts of the input when making a prediction. Instead of treating all inputs equally, the model computes attention weights that measure how important each element (for example, each word in a sentence) is for the current step. This makes the model better at capturing long-range relationships and understanding context efficiently.

1.1.2 Network architectures

Neuron

As it was stated, these networks are called *neural*, because they are loosely inspired by neuroscience. Basic computational unit of the network is the *neuron*. It is typically a function mapping a vector of inputs $\mathbf{x} = (x_1, \dots, x_n)$ into an output y by computing a weighted sum of the inputs plus a bias and then applying an activation function f :

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right),$$

where $w_1, w_2, \dots, w_n$ are the neuron's weights and b is the bias.

A Little Diagram

Architecture of the network

The architecture of a neural network describes how neurons and layers are organized, connected, and how they transform information. A network typically consists of an input layer that receives data, hidden layers that perform computations, and an output layer that produces predictions (see Figure 1.1).

Neurons in hidden layer work the same way as a perceptrons, except that instead of receiving the input vector, they take the outputs of neurons from the previous layer. Because of this similarity those networks are also called *multilayer perceptron*. When the information flows strictly in one direction - from input layer to output layer, we refer to the architecture as *feedforward network*. On the contrary, it is common for all neurons in the hidden layers to share the same activation function,

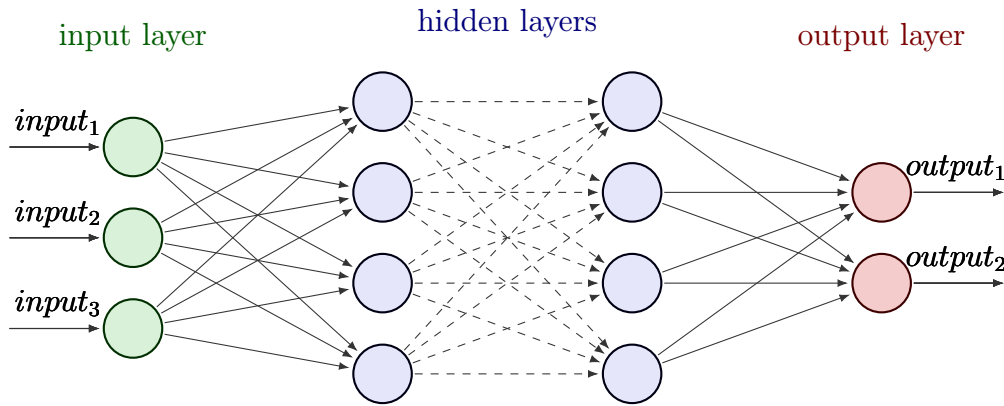


Figure 1.1: Scheme of the Neural Network

though this is not required; the choice of activation functions will be discussed in the [Subsection XXX](#). Likewise, the hidden layers do not need to have the same number of neurons, as it is illustrated in [Figure 1.2](#).

The flow of signals in networks does not have to be strictly one-directional. In some cases, we want to process a sequence of inputs with respect to their order. For such sequential data *recurrent networks* are used, in which neurons are able to store information from earlier signal passes and take it into account during future activations. It is implemented by creating loops between neurons to allow information to persist across time steps. These properties are essential for tasks such as time series modeling or human speech recognition.

It is also not necessary for neural networks to use a fully connected architecture. A partially connected structure is used in *convolutional networks* (see [Figure 1.2](#)), mainly applied to image and character recognition. Their idea is to alternate *convolutional* and *pooling layers*: the input is broken into small patches, which are processed through repeated layers. Early layers detect simple patterns like edges, while later ones combine them into more complex shapes. Because the connections are sparse, training is faster, and with the right design, convolutional networks can achieve very strong performance in image recognition.

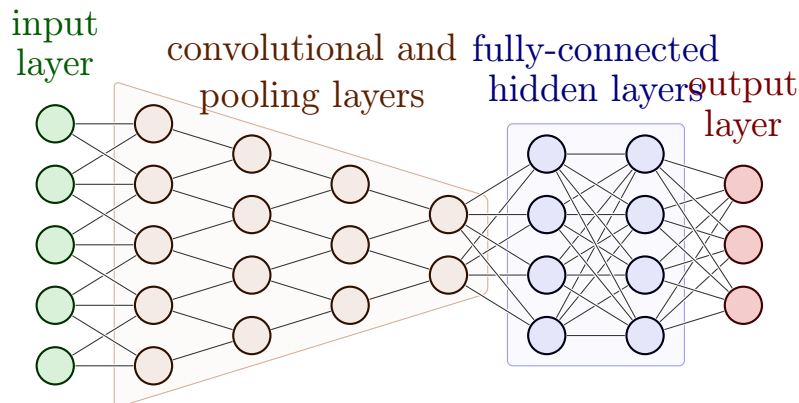


Figure 1.2: Scheme of the Convolutional Neural Network

The architecture of a network is therefore determined by the number of hidden layers, their width (the number of neurons in each layer), the way the neurons are connected, and the manner in which the signal flows.

Notation [introduce notation](#)

1.1.3 Notation

The activation of the j -th neuron in the i -th layer is computed as a weighted sum of the inputs from the previous layer plus a bias, passed through an activation function:

$$a_j^{(i)} = \sigma \left(\sum_{k=1}^n w_{j,k}^{(i)} a_k^{(i-1)} + b_j^{(i)} \right) = \sigma \left(\mathbf{w}_j^{(i)} \cdot \mathbf{a}^{(i-1)} + b_j^{(i)} \right),$$

where $\mathbf{w}_j^{(i)} = (w_{j,1}^{(i)}, w_{j,2}^{(i)}, \dots, w_{j,n}^{(i)})$ is the weight vector of the j -th neuron, $\mathbf{a}^{(i-1)} = (a_1^{(i-1)}, a_2^{(i-1)}, \dots, a_n^{(i-1)})^\top$ is the input vector from the previous layer, and $b_j^{(i)}$ is the bias term.

For the entire layer of m neurons, the activations can be written in vector form:

$$\begin{pmatrix} a_1^{(i)} \\ a_2^{(i)} \\ \vdots \\ a_m^{(i)} \end{pmatrix} = \sigma \left(\begin{pmatrix} w_{1,1}^{(i)} & w_{1,2}^{(i)} & \dots & w_{1,n}^{(i)} \\ w_{2,1}^{(i)} & w_{2,2}^{(i)} & \dots & w_{2,n}^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1}^{(i)} & w_{m,2}^{(i)} & \dots & w_{m,n}^{(i)} \end{pmatrix} \begin{pmatrix} a_1^{(i-1)} \\ a_2^{(i-1)} \\ \vdots \\ a_n^{(i-1)} \end{pmatrix} + \begin{pmatrix} b_1^{(i)} \\ b_2^{(i)} \\ \vdots \\ b_m^{(i)} \end{pmatrix} \right).$$

In compact vector-matrix notation, this can be written as:

$$\mathbf{a}^{(i)} = \sigma \left(\mathbf{W}^{(i)} \mathbf{a}^{(i-1)} + \mathbf{b}^{(i)} \right),$$

where $\mathbf{W}^{(i)}$ is the weight matrix connecting layer $i-1$ to layer i , $\mathbf{a}^{(i-1)}$ is the input vector, $\mathbf{b}^{(i)}$ is the bias vector, and $\mathbf{a}^{(i)}$ is the vector of activations in layer i .

This illustrates that the operation of a neural network layer can be equivalently expressed in element-wise form, summation form, or vector-matrix form.

1.1.4 Activation functions in the Hidden Layer

Activation function introduce non-linearity into model to learn complex data patterns. Without it, even a deep neural network would behave like a simple linear regression model. [aproximation of general functions by neural networks](#)

The earliest neural networks relied on smooth, bounded activation functions such as the *sigmoid* and *hyperbolic tangent* (*tanh*). The sigmoid function:

$$\phi(x) = \frac{1}{1 + e^{-x}}$$

compresses the input to the interval $(0, 1)$ and has a simple derivative:

$$\phi'(x) = \phi(x)(1 - \phi(x)),$$

which makes effective computation during *backpropagation algorithm*, which is discussed in [Section xx](#)

However, for very large positive or negative inputs, the sigmoid function approaches its asymptotes, causing its derivative to become very small. This can lead to extremely small gradients, contributing to the well-known *vanishing gradient problem* [8]. A similar issue arises with the tanh function

$$\phi(x) = \tanh(x) = \frac{1 + e^{-2x}}{1 - e^{-2x}}.$$

A major breakthrough came with the introduction of the *ReLU (Rectified Linear Unit)* defined as

$$\phi(x) = \max(\{0, x\}).$$

ReLU is computationally more efficient than sigmoidal functions, avoids saturation for positive inputs, and converges faster in deep architectures [12].

Main problem with ReLU as activation function is that in learning process it is prone to creating so-called *dead neurons*. It is a phenomenon, when its pre-activation value $w^T x + b$ consistently falls into the non-positive region, causing its output to remain zero for given inputs during a training process. Since their gradient is always zero, their weights stop updating and they no longer contribute to learning.

To cope with such issue, several variants have been proposed. *Leaky ReLU* uses small slope for negative inputs $\phi(x) = \max(\alpha x, x)$, which prevents complete inactivity. Another widely used alternative is the *Gaussian Error Linear Unit (GELU)*. Hendrycks and Gimpel in [11] showed that its smooth, probabilistic, Gaussian-based formulation improves performance across a variety of tasks. GELU has become a standard choice in transformer-based architectures, including models such as GPT [13] and BERT [14].

1.1.5 Activation Functions in Output Layer

The choice of activation function in the output layer of a neural network depends on the type of task and the desired output representation. For regression tasks, where the network predicts continuous values, the *identity function* $\phi(x) = x$ is commonly used.

For binary classification, the sigmoid function [Section XX](#) is frequently used. Its output lies in the interval $(0, 1)$, which can be interpreted as the probability of belonging to a positive class. Its differentiability enables efficient gradient-based learning.

For multiclass classification, the *softmax* function is commonly used. Given an input vector $\mathbf{x} = (x_1, x_2, \dots, x_k)^T \in \mathbb{R}^k$, softmax maps it to a probability distribution over k classes:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^k e^{x_j}}, \quad i = 1, \dots, k.$$

This ensures that $\text{softmax}(x_i) > 0$ for all i and $\sum_{i=1}^k \text{softmax}(x_i) = 1$, allowing the outputs to be interpreted as class probabilities. For this transformation to work, the last hidden layer must have the same number of neurons as the output layer, and its outputs are fed directly into the softmax, with no additional weights.

1.1.6 Loss Functions

Loss function quantifies the difference between a neural network's prediction $\hat{y}$ and target value y . It can be used to both evaluate performance of the model, and for a guidance in gradient-based learning, as it will be illustrated in the following subsection. Our goal is to make our predictions the most accurate and thus to minimize the loss function.

The choice of loss function depends on the type of task. For regression tasks, the *mean squared error* is commonly used:

$$\mathcal{L} = \text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

For binary classification, the *binary cross-entropy* loss is used, which is the negative log-likelihood of Bernoulli distribution:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right],$$

where $\hat{y}_i$ represents the predicted probability of the positive class.

For multiclass classification, the *categorical cross-entropy loss* is used in combination with the softmax activation:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^k y_{i,k} \log \hat{y}_{i,k},$$

where $y_{i,k} = 1$ if sample i belongs to class k and 0 otherwise, and $\hat{y}_{i,k}$ is the predicted probability for class k .

1.1.7 Gradient Descent

Gradient descent is an optimization algorithm used to train neural networks by minimizing loss function. Its main idea is iteratively adjusting the parameters of the model in the direction that reduces the loss the most. Formally, for a set of parameters $\boldsymbol{\theta}$ and a differentiable loss function $\mathcal{L}(\boldsymbol{\theta})$, the update rule naturally is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t),$$

where η is called *learning rate* controlling step size, and $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$ is the gradient of loss function with respect to all parameters at iteration t .

Under conditions such as convexity and Lipschitz-continuity of gradients, convergence to the global minimum is guaranteed. However, these conditions are rarely

satisfied in practice and thus the algorithm may converge to local minimum, diverge or oscillate.

There are multiple ways in which the parameters can be updated at each step, depending on the choice of *batch* - subset of training data used to estimate the gradient. In *batch gradient descent*, the gradient is computed over the entire dataset:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{1}{n} \sum_{i=1}^n \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t, y_i),$$

where n is the total number of training samples and $\mathcal{L}(\boldsymbol{\theta}_t, y_i)$ is the loss for a single sample. This provides an accurate update but can be computationally expensive for large datasets.

In *stochastic gradient descent*, the parameters are updated using a single randomly selected training sample per iteration:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t, y_i),$$

which introduces stochasticity, potentially helping the optimizer escape shallow local minima.

Mini-batch gradient descent updates the parameters using a small subset of size B , where $B \in \mathbb{N}$ and $1 < B < N$:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{1}{B} \sum_{i \in \text{mini-batch}} \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t, y_i),$$

This approach is the most commonly used in practice, because of its computational efficiency and smoother convergence. [15]

It is important to mention the *backpropagation algorithm*, which is used for effective computation of gradient of a loss function. It applies the chain rule of calculus to propagate from the output layer backward through the network and iteratively computing weights and biases. This reduces the overall computational complexity from exponential to linear with regard in the number of layers [8].

For effective learning, it is crucial to choose appropriate learning rate. It controls magnitude of each step in gradient-descent. A small learning rate can cause slow convergence. Conversely, a too large learning rate can cause divergence, or oscillatory behavior. Thus, selecting a fixed learning rate can cause instability or excessively slow convergence during training. Several modifications of learning rate are proposed. One of the most simple is an *exponential decay*:

$$\eta_t = \eta_0 e^{-kt},$$

where η_0 denotes the initial learning rate, $k > 0$ is the decay coefficient, and t is an iteration index.

Exponential decay has some limitations, as it reduces the learning rate uniformly without considering the specific behavior of individual parameters. The most effective method are so called adaptive algorithms, such as *Adam*, *RMSProp*, and *Adagrad*. **GLOSSARY** These methods automatically adjust the step size for each

parameter based on the history of gradients - this makes them useful when the gradients of some parameters are steep while others are shallow. Among them, Adam has become the most widely used optimizer due to its strong empirical performance, robustness across a variety of architectures, and relatively low sensitivity to the choice of the initial learning rate.

1.2 Introduction to Feature Selection

One of the fundamental motivations for feature selection is to deal with a *curse of dimensionality*, which refers to problems that arise, when analyzing high-dimensional data. As the number of features rises, distances between points become less informative, and many algorithms become less effective, or computationally more expensive.

As illustrated by the authors in [5]; consider the task of locating the nearest neighbor in p -dimensional unit hypercube. To capture a fixed fraction r of all observations, one expands a hypercube centered at the target point so that its edge length becomes $e_p(r) = r^{1/p}$. As shown in Figure 1.3, this neighborhood remains small in low dimensions but grows rapidly as p increases. For example, in ten dimensions, capturing only 1% or 10% of the data already requires covering about 63% and 80% of each variable's range. This shows, that two data points, which are close in a 2-dimensional space might be very distant in a 100-dimensional space. This phenomenon is referred to as *sparse data*, when most of the space contains few or no observations, leaving many combinations of feature values unrepresented in large portion of space. In classification tasks, sparse data can make it more difficult to predict the class of unseen observations.

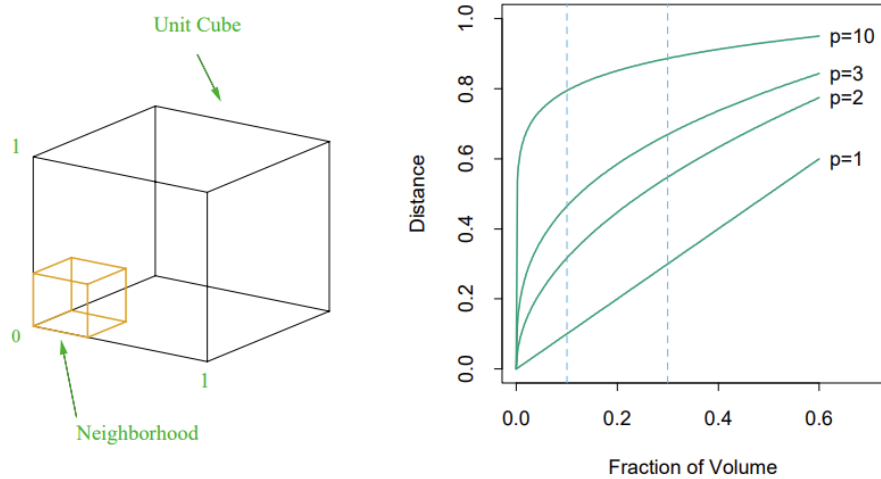


FIGURE 2.6. The curse of dimensionality is well illustrated by a subcubical neighborhood for uniform data in a unit cube. The figure on the right shows the side-length of the subcube needed to capture a fraction r of the volume of the data, for different dimensions p . In ten dimensions we need to cover 80% of the range of each coordinate to capture 10% of the data.

Figure 1.3: Illustration of the effect of dimensionality on nearest-neighbour regions.

The number of features determines a *hypothesis space*, the set of all functions a learning algorithm can choose to map inputs to outputs, and this space grows exponentially as the number of features increases [8, 6].

For instance, with n binary features and a binary class, the size of the hypothesis space is 2^{2^n} . For non-binary features, the size of the hypothesis space grows even more rapidly. For instance, if each feature can take m values and the class has l possible labels, the hypothesis space size becomes l^{m^n} , which further emphasizes the importance of feature selection.

Hypothesis space of a linear regression

For a fixed-size training dataset, reducing the number of features effectively reduces the number of required training instances. For example, if the number of binary features is reduced from $n = 10$ to $n = 5$, and the total number of training instances m is kept same, the ratio of training instances to possible input combinations, $m/2^n$, increases exponentially, effectively increasing the coverage of the input space [9].

It was demonstrated by Liu and Motoda [9], feature selection can significantly reduce the hypothesis space by eliminating irrelevant and redundant features, and a smaller hypothesis space makes it easier to identify plausible hypotheses.

Removal of the noise

[7] Definition of relevant and irrelevant features

Feature selection is process of selecting the most relevant features. Key aspects of feature selection include the selection criterion of the method, evaluation approach, and in some cases search strategy (mainly wrapper methods). Based on the selection criterion, most of the feature selection methods belong to one of these ¹three categories:

1. **Filter methods:** These perform feature selection independently of any predictive model. They rely on their intrinsic properties, such as correlation, χ^2 tests, or ANOVA [GLOSSARY](#) to rank the variables. Their main advantage is computational speed; however, they do not account for more complex interactions between features, such as non-linear or higher-order dependencies, that might influence the target variable. [Section XXs](#)
2. **Wrapper methods:** These are a category of feature selection techniques that evaluate subsets of features by training a machine learning model and measuring its performance. They treat the model as a black box and use it to find the combination of features that leads to the best prediction performance. Their main drawback is high computational demand, since the model must be trained multiple times for different feature subsets. [Section XXs](#) [GLOSSARY](#)
3. **Embedded methods:** In these methods, feature selection is integrated directly into the model training process. Examples include regularized linear models (e.g., LASSO or Ridge) and tree-based models that provide measures of *feature importance* such as the *Gini coefficient* or *entropy*. [Reference to book or chapter](#)

¹Some methods may not fit into these three categories.

feature selection improves interpretability and significantly reduce training time..

1.3 Generalization in Deep Neural Networks

For deep neural networks, the hypothesis space is typically extremely large due to the multiple layers and non-linear activation functions.

For example, consider the task of recognizing handwritten digits from the MNIST dataset, which contains 70,000 grayscale images of digits 0–9, and with image dimension of 28×28 .

A fully connected neural network with one hidden layer of 128 neurons can be used for this task. The hypothesis space - the set of all functions the network can represent - is determined by its parameters. With 784 input neurons, 128 hidden neurons, and 10 output neurons, the network has

$$\underbrace{784 \times 128}_{\text{input-to-hidden weights}} + \underbrace{128}_{\text{hidden biases}} + \underbrace{128 \times 10}_{\text{hidden-to-output weights}} + \underbrace{10}_{\text{output biases}} \approx 100,000 \text{ parameters.}$$

Each combination of these values defines a distinct hypothesis, illustrating that even a relatively simple architecture can have a large expressive *capacity*, which is its ability to represent a wide range of functions or patterns in the data.

Mention XOR problem

However, this large hypothesis space can lead to poor generalization. *Generalization* is the ability of a model to perform well on new, unseen data, not just on the data it was trained on. It reflects how well the model captures the underlying patterns rather than memorizing noise.

When the model fits the training data too poorly it said to underfit. *Underfitting* occurs when the chosen hypothesis is too simple relative to the data distribution; this prevents the model from capturing essential patterns and leads to high *training error*. **Training error definition, formula**

Figure illustrating Overfitting and underfitting

Conversely, when the model captures not only essential patterns, but also noise, it is said to overfit. *Overfitting* arises when the hypothesis is more complex than the data on which the model was trained and can lead to high *generalization error*, or also called *test error*. **Establish noise and test error**

To evaluate generalization, the available data is typically divided into separate subsets. The training set is used to fit the model and adjust its parameters, while the test set (or validation set) is kept aside to estimate the model's performance on unseen data. Measuring the error on the test set provides an estimate of the generalization error, helping to identify underfitting or overfitting of the model. **Training set, Validation set, test set in more detail**

Give illustative example of underfitting and overfitting

Add figure-graph showing relation of training and generalization error with regard to number of features

Deep learning - page 145

Important concept tightly linked to overfitting and underfitting is the *bias-variance trade-off*. Let $\hat{f}(x)$ denote the estimator learned from data, and let $f(x)$ be the true underlying function we are estimating. The expected prediction error at a point x can be decomposed as

$$\mathbb{E}[(\hat{f}(x) - f(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{variance}} + \sigma^2,$$

where σ^2 is the irreducible noise in the data.

The bias term quantifies the error due to restrictive assumptions in the model. A model with high bias fails to approximate complex patterns and tends to underfit.

The variance, which underlies the problem of overfitting, increases with model complexity. As complexity grows, bias decreases while variance rises. A larger number of parameters allows the model to capture more complex relationships between predictors and the response.

Neural networks, because of their high *capacity* and extremely large hypothesis space, can achieve low bias but often exhibit large variance, especially when the number of parameters is high relative to the sample size.

For this reason, the goal in practice is to find an appropriate balance:

$$\text{Generalization Error} \approx \text{Bias}^2 + \text{Variance} + \sigma^2.$$

Achieving this balance typically requires the use of *regularization techniques*, which will be discussed in [subsection X.x](#). For a model with parameters θ , weight decay (also known as L_2 -regularization) adds a penalty $\lambda \|\theta\|_2^2$ to the loss function, and thus effectively reducing variance:

$$L_{\text{reg}}(\theta) = L(\theta) + \lambda \|\theta\|_2^2.$$

Similarly, methods such as dropout, early stopping, and cross-validation help constrain model complexity without sacrificing too much expressive power. These strategies collectively aim to minimize expected test error by controlling variance while keeping bias acceptably low.

1.4 Evaluation of Model Performance

In this section, we introduce commonly used metrics for evaluating the performance of machine learning models.

1.4.1 Criterias for classification tasks

For classification models, the goal is to assign new samples to their appropriate categories. One of the most commonly used metrics in classification is *accuracy*. Accuracy represents the proportion of correctly classified samples out of all samples. Although accuracy provides an intuitive measure of performance, it can be misleading when class distributions are imbalanced.

For example, consider a disease with a low prevalence, such as 0.5%, and assume that healthy cases are assigned to the negative class, while disease cases are assigned to the positive class. If we create a naive model that classifies every observation as negative, the accuracy of this model would be 99.5%. At first glance, this might appear to be a model with excellent predictive performance. However, this high accuracy is misleading, as the model fails completely to identify people with a disease. This example illustrates that accuracy alone can be insufficient. For this reason, additional metrics are used that better capture performance across different types of errors.

In classification tasks solved by neural networks, the model typically outputs a vector of predicted probabilities obtained through the *softmax* or *sigmoid* activation function. To transform these probability scores into one class decision, a decision threshold, or also called *cut-off* point must be specified. In binary classification, a default threshold is 0.5, but by adjusting this value, we can better suit the model to the requirements of a specific application. In multinomial classification usually an observation is assigned with the class with highest predicted probability.

In the theory of binary classification, predictions can be divided into four distinct categories:

- **True Positive (TP)**: a sample from the positive class that has been correctly classified as positive.
- **True Negative (TN)**: a sample from the negative class that has been correctly classified as negative.
- **False Positive (FP)**: a sample from the negative class that has been incorrectly classified as positive, also called a *Type I error*.
- **False Negative (FN)**: a sample from the positive class that has been incorrectly classified as negative, also called a *Type II error*.

The results of a classification model are often summarized using a *confusion matrix*, as depicted in 1.1:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive	False Negative (Type II Error)
Actual Negative	False Positive (Type I Error)	True Negative

Table 1.1: Confusion matrix.

From the confusion matrix, several performance measures can be derived:

- **Precision** – measures the proportion of correctly predicted positive samples among all predicted positive samples:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity)** – measures the proportion of correctly identified positive samples out of all actual positives:

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Specificity** – measures the proportion of correctly identified negative samples out of all actual negatives:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- **F1 score** – the harmonic mean of precision and recall:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Among these characteristics, the *true positive rate (TPR)*, also known as sensitivity, represents the proportion of correctly identified positive samples, while the *false positive rate (FPR)*, defined as $1 - \text{specificity}$, quantifies the proportion of negative samples incorrectly classified as positive.

Although metrics, that have been already mentioned can provide valuable information about model performance at a specific classification threshold, they do not capture how the model behaves across all possible thresholds. To address this problem, methods such as the *ROC curve* and the *PR curve* are used.

If we plot, for different values of cut-off points $p \in [0, 1]$, a graph of the corresponding FPR values on the x -axis and TPR values on the y -axis, we obtain the so-called *Receiver Operating Characteristic (ROC) curve*. An example of an ROC curve can be seen in [Figure X.X](#). Since the ROC curve reflects both TPR and FPR across all possible choices of the cut-off point p , it represents a robust measure of the quality of binary classifiers. The most widely used scalar summary of the ROC curve is the *Area Under the Curve (AUC)*, which corresponds to the total area lying below it. Clearly, $AUC \in [0, 1]$, and higher values indicate a better classifier.

In addition to the full AUC, it is often useful to evaluate the classifier only on a relevant portion of the ROC curve. This leads to the concept of the *partial area under the ROC curve (pAUC)*. The pAUC measures the area under the ROC curve over a restricted range of FPR values, typically chosen to reflect the region where the classifier is expected to operate in practice. For example, in applications where a very low false positive rate is required, one may compute the pAUC over the interval $FPR \in [0, \alpha]$ for a small α .

[youden index](#)

[Introduce AUPR](#)

[Entropy, cross-entropy, KL - divergence](#)

1.4.2 Criteria for Regression Tasks

In regression tasks, we are supposed to predict the target variable which is in the continuous form. To assess prediction performance, several commonly used metrics

can be used. Let y_i denote the observed value of the dependent variable, $\hat{y}_i$ the predicted value, and n the number of observations.

Mean Squared Error (MSE) measures the average squared difference between observed and predicted values. Because the errors are squared, it penalizes larger errors more strongly than the smaller ones. For this reason, MSE is widely used as a training loss function.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) is the square root of MSE. It is expressed in the same units as predicted variable, which makes interpretation more intuitive. It, however, remains sensitive to outliers.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Mean Absolute Error (MAE) calculates the average absolute difference between observed and predicted values. Unlike MSE, it is less affected by extreme observations and provides a more robust measure of typical error size.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

R^2 represents the proportion of variance in the dependent variable explained by the model. Typically, $R^2 \in [0, 1]$, with higher values indicating a better fit. Although commonly used, it tends to increase as more predictors are added, even if they do not contribute meaningful information; however, adjusted versions of the metric can be used to account for model complexity. R^2 is common in linear models but not in neural networks, where MSE or RMSE better evaluate prediction performance and are suitable as a loss function.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

In the context of neural networks, metrics such as MSE and RMSE are not only useful for evaluating overall prediction performance, but also guiding the training process. They are often used as a loss functions during network optimization, helping adjust weights and biases to minimize prediction error.

1.4.3 architectures, crossvalidation, python pipeline, grid-search, hyperparameters

DIVIDING DATA INTO 3 CATEGORIES

Chapter 2

Feature Selection Methods

Search strategies

A *search strategy* defines the order in which the variable subsets are evaluated. These strategies are particularly important for wrapper methods. Few of the search strategies are proposed by Guyon et al.[10]. Most common strategies are forward selection, backward elimination, floating search, branch-and-bound, and randomized search. *Forward selection* starts with no features and adds them one by one, keeping those that improve the model the most. *Backward elimination* starts with all features and removes them one by one, discarding those that contribute least.

Both methods are considered exhaustive searches, as they evaluate all possible subsets of features up to a certain size. For datasets with many features, forward and backward selection can be computationally expensive. In forward selection, if the optimal number of features k is unknown, one may need to evaluate all subsets of size 1 to k , requiring $\binom{N}{1} + \binom{N}{2} + \dots + \binom{N}{k}$ combinations, which gives a time complexity of $O(2^N)$.

For this reason, more efficient algorithms have been developed, such as sequential forward selection (SFS) and sequential backward selection (SBS). SFS starts with an empty feature set and iteratively adds the feature, that yields the greatest improvement in model performance, while previously selected features remain selected. SBS is similar to SFS, but it starts with the full feature set and iteratively removes the feature that contributes the least. Both SFS and SBS are heuristic, thus they do not guarantee the optimal subset. Research also shows that exhaustive search is more prone to overfitting, than sequential methods. [\[add citation or reference\]](#)

Floating methods are an extension of sequential selection methods that allow conditional backtracking: Sequential Forward Floating Selection (SFFS) can remove previously added features if they later become redundant, and Sequential Backward Floating Selection (SBFS) can re-add previously removed features, making the search more flexible and often more accurate than simple SFS or SBS.

Branch and bound strategies are systematic search methods used to find the optimal subset of features without evaluating all possibilities. Branch and bound relies on monotonicity, but this assumption is rarely satisfied in deep neural networks, making it less effective than heuristic methods.

Chapter 3

Feature Selection Methods

Search strategies

A *search strategy* defines the order in which the variable subsets are evaluated. These strategies are particularly important for wrapper methods. Few of the search strategies are proposed by Guyon et al.[10]. Most common strategies are forward selection, backward elimination, floating search, branch-and-bound, and randomized search. *Forward selection* starts with no features and adds them one by one, keeping those that improve the model the most. *Backward elimination* starts with all features and removes them one by one, discarding those that contribute least.

Both methods are considered exhaustive searches, as they evaluate all possible subsets of features up to a certain size. For datasets with many features, forward and backward selection can be computationally expensive. In forward selection, if the optimal number of features k is unknown, one may need to evaluate all subsets of size 1 to k , requiring $\binom{N}{1} + \binom{N}{2} + \dots + \binom{N}{k}$ combinations, which gives a time complexity of $O(2^N)$.

For this reason, more efficient algorithms have been developed, such as sequential forward selection (SFS) and sequential backward selection (SBS). SFS starts with an empty feature set and iteratively adds the feature, that yields the greatest improvement in model performance, while previously selected features remain selected. SBS is similar to SFS, but it starts with the full feature set and iteratively removes the feature that contributes the least. Both SFS and SBS are heuristic, thus they do not guarantee the optimal subset. Research also shows that exhaustive search is more prone to overfitting, than sequential methods. [\[add citation or reference\]](#)

Floating methods are an extension of sequential selection methods that allow conditional backtracking: Sequential Forward Floating Selection (SFFS) can remove previously added features if they later become redundant, and Sequential Backward Floating Selection (SBFS) can re-add previously removed features, making the search more flexible and often more accurate than simple SFS or SBS.

Branch and bound strategies are systematic search methods used to find the optimal subset of features without evaluating all possibilities. Branch and bound relies on monotonicity, but this assumption is rarely satisfied in deep neural networks, making it less effective than heuristic methods. Nunc velit augue, scelerisque dignissim, lobortis et, aliquam in, risus. In eu eros. Vestibulum ante ipsum primis

in faucibus orci luctus et ultrices posuere cubilia Curae; Curabitur vulputate elit viverra augue. Mauris fringilla, tortor sit amet malesuada mollis, sapien mi dapibus odio, ac imperdiet ligula enim eget nisl. Quisque vitae pede a pede aliquet suscipit. Phasellus tellus pede, viverra vestibulum, gravida id, laoreet in, justo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer commodo luctus lectus. Mauris justo. Duis varius eros. Sed quam. Cras lacus eros, rutrum eget, varius quis, convallis iaculis, velit. Mauris imperdiet, metus at tristique venenatis, purus neque pellentesque mauris, a ultrices elit lacus nec tortor. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent malesuada. Nam lacus lectus, auctor sit amet, malesuada vel, elementum eget, metus. Duis neque pede, facilisis eget, egestas elementum, nonummy id, neque.

Proin non sem. Donec nec erat. Proin libero. Aliquam viverra arcu. Donec vitae purus. Donec felis mi, semper id, scelerisque porta, sollicitudin sed, turpis. Nulla in urna. Integer varius wisi non elit. Etiam nec sem. Mauris consequat, risus nec congue condimentum, ligula ligula suscipit urna, vitae porta odio erat quis sapien. Proin luctus leo id erat. Etiam massa metus, accumsan pellentesque, sagittis sit amet, venenatis nec, mauris. Praesent urna eros, ornare nec, vulputate eget, cursus sed, justo. Phasellus nec lorem. Nullam ligula ligula, mollis sit amet, faucibus vel, eleifend ac, dui. Aliquam erat volutpat.

Fusce vehicula, tortor et gravida porttitor, metus nibh congue lorem, ut tempus purus mauris a pede. Integer tincidunt orci sit amet turpis. Aenean a metus. Aliquam vestibulum lobortis felis. Donec gravida. Sed sed urna. Mauris et orci. Integer ultrices feugiat ligula. Sed dignissim nibh a massa. Donec orci dui, tempor sed, tincidunt nonummy, viverra sit amet, turpis. Quisque lobortis. Proin venenatis tortor nec wisi. Vestibulum placerat. In hac habitasse platea dictumst. Aliquam porta mi quis risus. Donec sagittis luctus diam. Nam ipsum elit, imperdiet vitae, faucibus nec, fringilla eget, leo. Etiam quis dolor in sapien porttitor imperdiet.

3.1 Filter Methods

Donec ac velit. Sed convallis vestibulum sapien. Vivamus tempor lacus sed lacus. Nunc ut lorem. Ut et tortor. Nullam varius wisi at diam. Etiam ultricies, dolor sit amet fermentum vulputate, neque libero vestibulum orci, vitae fringilla neque arcu aliquet ante. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque venenatis lobortis augue. Sed tempor, tellus iaculis pellentesque pharetra, pede dui malesuada mauris, vel ultrices urna mauris ac nibh. Etiam nibh odio, ultricies vehicula, vestibulum vitae, feugiat eleifend, felis. Vivamus pulvinar. Aliquam erat volutpat. Nulla egestas venenatis metus. Nam feugiat nunc quis elit egestas sagittis. Sed vitae felis. In libero arcu, rhoncus in, commodo eget, auctor in, enim. Vivamus suscipit est. Nulla dapibus, magna vel aliquet egestas, massa massa hendrerit lacus, ac rutrum tellus tellus sit amet felis. Cras viverra.

Suspendisse eu nunc. Aliquam dignissim urna sit amet mauris. Cras commodo, urna ut porttitor venenatis, arcu metus sodales risus, vitae gravida sapien ligula in est. Donec vulputate sollicitudin wisi. Donec vehicula, est id interdum ornare, nibh tellus consectetur justo, a ultrices felis erat at lectus. In est massa, malesuada non,

suscipit at, ullamcorper eu, elit. Nam nulla lacus, bibendum sit amet, sagittis sed, tempor eget, libero. Praesent ligula. Suspendisse nulla. Etiam diam. Nulla ante diam, vestibulum et, aliquet ac, imperdiet vitae, urna. Fusce tincidunt lacus vel elit. Maecenas dictum, tortor non euismod bibendum, pede nibh pretium tellus, at dignissim leo eros eget pede. Nulla venenatis eleifend eros. Aenean ut odio dignissim augue rutrum faucibus. Fusce posuere, tellus eget viverra mattis, erat tellus porta mi, at facilisis sem nibh non urna. Phasellus quis turpis quis mauris suscipit vulputate. Sed interdum lacus non velit. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae.

Vivamus vehicula leo a justo. Quisque nec augue. Morbi mauris wisi, aliquet vitae, dignissim eget, sollicitudin molestie, ligula. In dictum enim sit amet risus. Curabitur vitae velit eu diam rhoncus hendrerit. Vivamus ut elit. Praesent mattis ipsum quis turpis. Curabitur rhoncus neque eu dui. Etiam vitae magna. Nam ullamcorper. Praesent interdum bibendum magna. Quisque auctor aliquam dolor. Morbi eu lorem et est porttitor fermentum. Nunc egestas arcu at tortor varius viverra. Fusce eu nulla ut nulla interdum consectetur. Vestibulum gravida. Morbi mattis libero sed est.

3.1.1 Subsection

3.2 Wrapper Methods

3.3 Embedded Methods

In accumsan convallis metus. Aenean est. Donec pharetra porta odio. Duis nunc nisl, imperdiet ac, tincidunt vitae, varius sit amet, felis. Curabitur wisi. Ut iaculis, nunc in lacinia egestas, elit enim tincidunt turpis, at luctus ipsum augue condimentum metus. Aenean lorem wisi, cursus sit amet, mollis nec, porta ac, augue. Vivamus massa. Praesent rhoncus imperdiet orci. Aenean pharetra dolor ut sapien. Maecenas egestas augue semper dolor.

Vestibulum at lectus. Vestibulum dapibus placerat magna. Suspendisse dolor urna, condimentum sit amet, euismod a, adipiscing a, enim. Aliquam erat volutpat. Donec imperdiet dolor non mi. Phasellus magna metus, dictum sit amet, laoreet non, dictum vel, dui. Suspendisse potenti. Nunc turpis risus, porta vel, pharetra id, eleifend vitae, justo. Duis pulvinar dolor sit amet urna. Integer eu eros. Nulla facilisi. Duis dui. Nullam vitae quam. Morbi a nunc in elit sodales euismod. Nunc sed orci. Etiam malesuada metus vitae felis. Suspendisse imperdiet velit in tellus.

Chapter 4

Practical application

Fusce tristique risus id wisi. Integer molestie massa id sem. Vestibulum vel dolor. Pellentesque vel urna vel risus ultricies elementum. Quisque sapien urna, blandit nec, iaculis ac, viverra in, odio. In hac habitasse platea dictumst. Morbi neque lacus, convallis vitae, commodo ac, fermentum eu, velit. Sed in orci. In fringilla turpis non arcu. Donec in ante. Phasellus tempor feugiat velit. Aenean varius massa non turpis. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae;

Aliquam tortor. Morbi ipsum massa, imperdiet non, consectetur vel, feugiat vel, lorem. Quisque eget lorem nec elit malesuada vestibulum. Quisque sollicitudin ipsum vel sem. Nulla enim. Proin nonummy felis vitae felis. Nullam pellentesque. Duis rutrum feugiat felis. Mauris vel pede sed libero tincidunt mollis. Phasellus sed urna rhoncus diam euismod bibendum. Phasellus sed nisl. Integer condimentum justo id orci iaculis varius. Quisque et lacus. Phasellus elementum, justo at dignissim auctor, wisi odio lobortis arcu, sed sollicitudin felis felis eu neque. Praesent at lacus.

Nunc velit augue, scelerisque dignissim, lobortis et, aliquam in, risus. In eu eros. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Curabitur vulputate elit viverra augue. Mauris fringilla, tortor sit amet malesuada mollis, sapien mi dapibus odio, ac imperdiet ligula enim eget nisl. Quisque vitae pede a pede aliquet suscipit. Phasellus tellus pede, viverra vestibulum, gravida id, laoreet in, justo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer commodo luctus lectus. Mauris justo. Duis varius eros. Sed quam. Cras lacus eros, rutrum eget, varius quis, convallis iaculis, velit. Mauris imperdiet, metus at tristique venenatis, purus neque pellentesque mauris, a ultrices elit lacus nec tortor. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent malesuada. Nam lacus lectus, auctor sit amet, malesuada vel, elementum eget, metus. Duis neque pede, facilisis eget, egestas elementum, nonummy id, neque.

Proin non sem. Donec nec erat. Proin libero. Aliquam viverra arcu. Donec vitae purus. Donec felis mi, semper id, scelerisque porta, sollicitudin sed, turpis. Nulla in urna. Integer varius wisi non elit. Etiam nec sem. Mauris consequat, risus nec congue condimentum, ligula ligula suscipit urna, vitae porta odio erat quis sapien. Proin luctus leo id erat. Etiam massa metus, accumsan pellentesque, sagittis sit

amet, venenatis nec, mauris. Praesent urna eros, ornare nec, vulputate eget, cursus sed, justo. Phasellus nec lorem. Nullam ligula ligula, mollis sit amet, faucibus vel, eleifend ac, dui. Aliquam erat volutpat.

Fusce vehicula, tortor et gravida porttitor, metus nibh congue lorem, ut tempus purus mauris a pede. Integer tincidunt orci sit amet turpis. Aenean a metus. Aliquam vestibulum lobortis felis. Donec gravida. Sed sed urna. Mauris et orci. Integer ultrices feugiat ligula. Sed dignissim nibh a massa. Donec orci dui, tempor sed, tincidunt nonummy, viverra sit amet, turpis. Quisque lobortis. Proin venenatis tortor nec wisi. Vestibulum placerat. In hac habitasse platea dictumst. Aliquam porta mi quis risus. Donec sagittis luctus diam. Nam ipsum elit, imperdiet vitae, faucibus nec, fringilla eget, leo. Etiam quis dolor in sapien porttitor imperdiet.

Cras pretium. Nulla malesuada ipsum ut libero. Suspendisse gravida hendrerit tellus. Maecenas quis lacus. Morbi fringilla. Vestibulum odio turpis, tempor vitae, scelerisque a, dictum non, massa. Praesent erat felis, porta sit amet, condimentum sit amet, placerat et, turpis. Praesent placerat lacus a enim. Vestibulum non eros. Ut congue. Donec tristique varius tortor. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Nam dictum dictum urna.

4.1 Section

In accumsan convallis metus. Aenean est. Donec pharetra porta odio. Duis nunc nisl, imperdiet ac, tincidunt vitae, varius sit amet, felis. Curabitur wisi. Ut iaculis, nunc in lacinia egestas, elit enim tincidunt turpis, at luctus ipsum augue condimentum metus. Aenean lorem wisi, cursus sit amet, mollis nec, porta ac, augue. Vivamus massa. Praesent rhoncus imperdiet orci. Aenean pharetra dolor ut sapien. Maecenas egestas augue semper dolor.

Vestibulum at lectus. Vestibulum dapibus placerat magna. Suspendisse dolor urna, condimentum sit amet, euismod a, adipiscing a, enim. Aliquam erat volutpat. Donec imperdiet dolor non mi. Phasellus magna metus, dictum sit amet, laoreet non, dictum vel, dui. Suspendisse potenti. Nunc turpis risus, porta vel, pharetra id, eleifend vitae, justo. Duis pulvinar dolor sit amet urna. Integer eu eros. Nulla facilisi. Duis dui. Nullam vitae quam. Morbi a nunc in elit sodales euismod. Nunc sed orci. Etiam malesuada metus vitae felis. Suspendisse imperdiet velit in tellus.

Nullam elit orci, condimentum vitae, accumsan quis, gravida non, velit. Morbi pellentesque accumsan elit. Aenean est purus, eleifend ac, dictum at, dignissim sed, dolor. Vestibulum volutpat sapien quis augue. Maecenas vulputate accumsan sapien. Nam mattis, lacus non iaculis aliquet, mi elit varius lectus, eu malesuada dolor nunc at wisi. Aliquam ligula. Mauris nisl elit, molestie vitae, gravida sit amet, facilisis convallis, enim. Sed urna. Praesent et augue. Fusce pellentesque. Maecenas varius orci eget nisl. Donec tempor rhoncus turpis. Integer nibh. Cras metus erat, tincidunt et, scelerisque quis, bibendum sed, dui. Suspendisse potenti.

Conclusion

Zde můžete napsat závěr. Zde můžete napsat závěr. Zde můžete napsat závěr. Zde
můžete napsat závěr. Zde můžete napsat závěr. Zde můžete napsat závěr. Zde
můžete napsat závěr. Zde můžete napsat závěr. Zde můžete napsat závěr. Zde
můžete napsat závěr. Zde můžete napsat závěr. Zde můžete napsat závěr. Zde
můžete napsat závěr. Zde můžete napsat závěr. Zde můžete napsat závěr. Zde
můžete napsat závěr.

Zde můžete napsat závěr. Zde můžete napsat závěr. Zde můžete napsat závěr.
Zde můžete napsat závěr. Zde napsat závěr. Zde můžete napsat závěr. Zde můžete
napsat závěr. Zde můžete napsat závěr. Zde můžete napsat závěr. Zde můžete
napsat závěr. Zde můžete napsat závěr. Zde můžete napsat závěr. Zde můžete
napsat závěr. Zde můžete napsat závěr.

Quisque facilisis auctor sapien. Pellentesque gravida hendrerit lectus. Mauris rutrum sodales sapien. Fusce hendrerit sem vel lorem. Integer pellentesque massa vel augue. Integer elit tortor, feugiat quis, sagittis et, ornare non, lacus. Vestibulum posuere pellentesque eros. Quisque venenatis ipsum dictum nulla. Aliquam quis quam non metus eleifend interdum. Nam eget sapien ac mauris malesuada adipiscing. Etiam eleifend neque sed quam. Nulla facilisi. Proin a ligula. Sed id dui eu nibh egetas tincidunt. Suspendisse arcu.

Maecenas dui. Aliquam volutpat auctor lorem. Cras placerat est vitae lectus. Curabitur massa lectus, rutrum euismod, dignissim ut, dapibus a, odio. Ut eros erat, vulputate ut, interdum non, porta eu, erat. Cras fermentum, felis in porta congue, velit leo facilisis odio, vitae consecetuer lorem quam vitae orci. Sed ultrices, pede eu placerat auctor, ante ligula rutrum tellus, vel posuere nibh lacus nec nibh. Maecenas laoreet dolor at enim. Donec molestie dolor nec metus. Vestibulum libero. Sed quis erat. Sed tristique. Duis pede leo, fermentum quis, consecetuer eget, vulputate sit amet, erat.

Donec vitae velit. Suspendisse porta fermentum mauris. Ut vel nunc non mauris pharetra varius. Duis consequat libero quis urna. Maecenas at ante. Vivamus varius, wisi sed egestas tristique, odio wisi luctus nulla, lobortis dictum dolor ligula in lacus. Vivamus aliquam, urna sed interdum porttitor, metus orci interdum odio, sit amet euismod lectus felis et leo. Praesent ac wisi. Nam suscipit vestibulum sem. Praesent eu ipsum vitae pede cursus venenatis. Duis sed odio. Vestibulum eleifend. Nulla ut massa. Proin rutrum mattis sapien. Curabitur dictum gravida ante.

Appendix

Sem můžete přidat přílohu. Pokud chcete “Přílohy”, tak upravte definici záhlaví v souboru `sci.muni.thesis.sty`, viz příkaz `\HlavickaPriloha`.

Quisque facilisis auctor sapien. Pellentesque gravida hendrerit lectus. Mauris rutrum sodales sapien. Fusce hendrerit sem vel lorem. Integer pellentesque massa vel augue. Integer elit tortor, feugiat quis, sagittis et, ornare non, lacus. Vestibulum posuere pellentesque eros. Quisque venenatis ipsum dictum nulla. Aliquam quis quam non metus eleifend interdum. Nam eget sapien ac mauris malesuada adipiscing. Etiam eleifend neque sed quam. Nulla facilisi. Proin a ligula. Sed id dui eu nibh egestas tincidunt. Suspendisse arcu.

Maecenas dui. Aliquam volutpat auctor lorem. Cras placerat est vitae lectus. Curabitur massa lectus, rutrum euismod, dignissim ut, dapibus a, odio. Ut eros erat, vulputate ut, interdum non, porta eu, erat. Cras fermentum, felis in porta congue, velit leo facilisis odio, vitae consectetur lorem quam vitae orci. Sed ultrices, pede eu placerat auctor, ante ligula rutrum tellus, vel posuere nibh lacus nec nibh. Maecenas laoreet dolor at enim. Donec molestie dolor nec metus. Vestibulum libero. Sed quis erat. Sed tristique. Duis pede leo, fermentum quis, consectetur eget, vulputate sit amet, erat.

Bibliography

- [1] S. J. Monaquiel a K. M. Schmidt, *On M -functions and operator theory for non-self-adjoint discrete Hamiltonian systems*, v “Special Issue: 65th birthday of Prof. Desmond Evans”, J.Comput. Appl. Math. **208** (2007), č. 1, 82–101.
- [2] M. Murata, *Positive solutions and large time behaviors of Schrödinger semi-groups, Simon’s problem*, J. Funct. Anal. **56** (1984), č. 3, 300–310.
- [3] R. Šimon Hilscher a P. Zemánek, *Friedrichs extension of operators defined by linear Hamiltonian systems on unbounded interval*, v “Equadiff 12”, Proceedings of the Conference on Differential Equations and their Applications (Brno, 2009), J. Diblík, O. Došlý, P. Drábek a E. Feistauer, editoři, Math. Bohem. **135** (2010), č. 2, 209–222.
- [4] W. T. Reid, *Sturmian Theory for Ordinary Differential Equations*, Applied Mathematical Sciences, Springer-Verlag, New York, 1980. ISBN 0-387-90542-1.
- [5] T. Hastie, R. Tibshirani, a J. Friedman, *The Elements of Statistical Learning*, Springer, 2001.
- [6] T. Mitchell, *Machine Learning*, McGraw–Hill, New York, 1997.
- [7] A. Blum a P. Langley, *Selection of relevant features and examples in machine learning*, Artificial Intelligence **97** (1997), 245–271.
- [8] I. Goodfellow, Y. Bengio a A. Courville, *Deep Learning*, MIT Press, London, 2016. ISBN 9780262035613.
- [9] H. Liu a H. Motoda, *Computational Methods of Feature Selection*, Chapman and Hall, Boca Raton, 2007. ISBN 9781584888789.
- [10] I. Guyon, M. Nikravesh, S. Gunn a L. Zadeh, *Feature Extraction: Foundations and Applications*, Springer, 2006.
- [11] D. Hendrycks a K. Gimpel, *Gaussian Error Linear Units (GELUs)*, 2016.
- [12] C. C. Aggarwal, *Neural Networks and Deep Learning*, Springer Publishing Company, 2018.

- [13] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, a I. Sutskever, *Language Models are Unsupervised Multitask Learners*, OpenAI, 2019, https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- [14] J. Devlin, M.-W. Chang, K. Lee, a K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, arXiv:1810.04805, 2018, <https://arxiv.org/abs/1810.04805>.
- [15] GeeksforGeeks, *Mini-Batch Gradient Descent in Deep Learning*, last updated 30 Sep 2025, <https://www.geeksforgeeks.org/mini-batch-gradient-descent-in-deep-learning/>.
- [16] D. O. Hebb, *The Organization of Behavior: A Neuropsychological Theory*, Wiley, New York, 1949. ISBN 9780471232312.
- [17] F. Rosenblatt, *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*, *Psychological Review* **65** (1958), 386–408.
- [18] M. Minsky a S. Papert, *Perceptrons: An Introduction to Computational Geometry*, MIT Press, Cambridge, 1969. ISBN 9780262630222.
- [19] Andrew L. Beam (Harvard Med. School), *Deep Learning 101: Neural Networks Are an Old Idea*, lecture slides, 2017. Available at: https://hms-dbmi.github.io/BMI_707/assets/files/bmi_707_lecture1.pdf
- [20] D. E. Rumelhart, G. E. Hinton a R. J. Williams, *Learning representations by back-propagating errors*, *Nature* **323** (1986), 533–536.
- [21] Y. LeCun, L. Bottou, Y. Bengio a P. Haffner, *Gradient-based learning applied to document recognition*, *Proceedings of the IEEE* **86** (1998), 2278–2324.
- [22] Y. LeCun, Y. Bengio a G. Hinton, *Deep Learning*, *Nature* **521** (2015), 436–444.
- [23] A. Krizhevsky, I. Sutskever a G. Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, *Advances in Neural Information Processing Systems* **25** (2012), 1097–1105.
- [24] A. Vaswani et al., *Attention Is All You Need*, *Advances in Neural Information Processing Systems* **30** (2017).

Index

cesky, 23

rea, 23

se, 19, 23

čeština v rejstříku, 3, 19, 23

ře, 23

še, 19, 23

